Bölüm 2: Oyun Kuramına Giriş

Algoritmik Oyun Kuramı

Algoritmik bakış | Oyunların sınıflandırılması | Klasik 2×2 oyunlar | Saf dengeler | Alt oyun mükemmel denge | Kırkayak oyunu

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

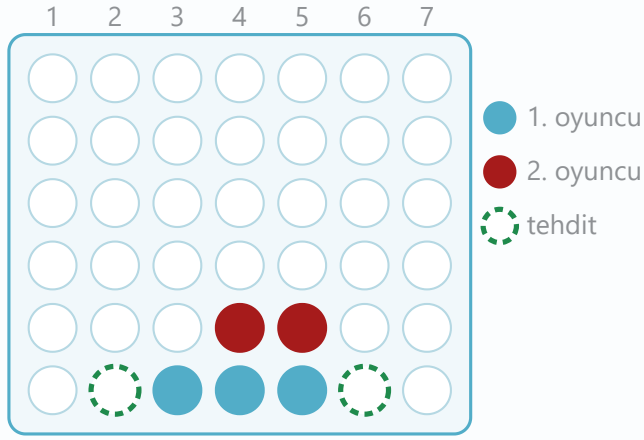
Oyunla Başlangıç: Connect Four

Oyunun kuralı

İki oyuncu sırayla yedi sütundan birine pul bırakır; pul o sütundaki en alt boş hücreye düşer ve yatay, dikey ya da çapraz dört pulu ilk dizen kazanır.

Merak sorusu

- İlk hamleyi yapan mı avantajlı, ikinci mi?
- İki oyuncu da hiç hata yapmasa oyunun sonucu önceden belli midir?



Sıra 2. oyuncuda: iki tehditten yalnızca birini kapatabilir.

Connect Four oyunu →

TEMEL KAVRAMLAR

Öğrenme Çıktıları

Algoritmik bakış

Algoritmik oyun kuramının dört sorusunu ve Pareto optimalliği bir örnek üzerinde açıklamak.

Sınıflandırma

Oyunları zamanlama, işbirliği, toplam kazanç, simetri, bilgi ve ufuk boyutlarında ayırt etmek.

Klasik oyunlar

Altı klasik 2×2 oyunun saf dengelerini en iyi tepkilerle bulmak; çatışma ve koordinasyon yapısını yorumlamak.

Geriye doğru çıkarım

Sıralı oyunları yapraktan köke (elle ve kodla) çözmek ve alt oyun mükemmel dengeyi yorumlamak.

TEMEL KAVRAMLAR

Algoritmik Oyun Kuramının Dört Sorusu

1. Hesaplama: denge bulunabilir mi?

Polinom zamanda çözülen problemler P sınıfındadır. Sıfır toplamlı oyunlar doğrusal programlamayla bu sınıftadır; genel oyunlarda Nash dengesi bulmak **PPAD-tam**, genelleştirilmiş n×n Hex ise **PSPACE-tam**dır.

2. Verimlilik: bencillik ne kaybettirir?

Anarşinin Bedeli = en kötü dengenin maliyeti / en iyi sonucun maliyeti. Mahkûmun İkileminde denge toplam 4 yıl, (Sessiz, Sessiz) 2 yıl: oran 4/2 = 2.

3. Tasarım: hangi kurallar?

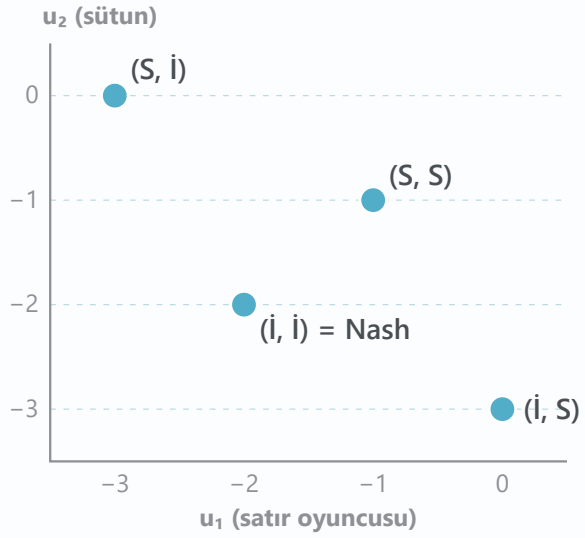
Mekanizma tasarımı, oyuncuların özel bilgisini dürüstçe bildirmesini sağlayan (teşvik uyumlu) kuralları arar: açık artırmalar, eşleştirme, oylama.

4. Dinamik: oyuncular dengeye nasıl ulaşır?

Tekrarlanan ve çevrimiçi oyunlarda bilgi zamanla gelir; oyuncular geçmişten öğrenerek stratejisini uyarlar.

Dört sorunun ortak dili: oyunların sınıflandırılması, klasik oyunların dengeleri ve sıralı oyunlarda geriye doğru çıkarım.

Pareto Optimallik: Kazanç Uzayında Bakış



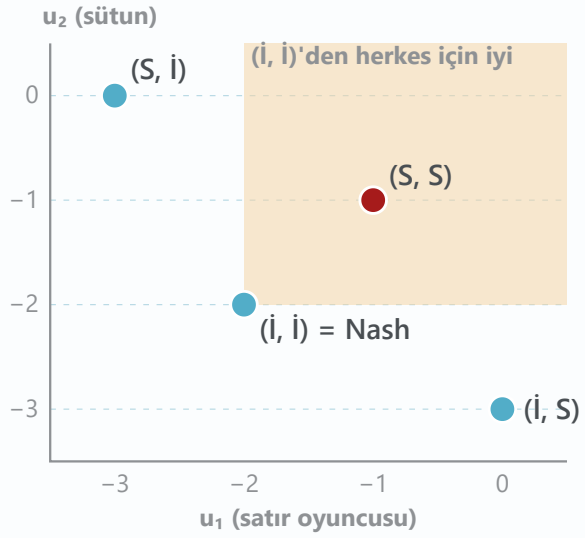
Her sonuç bir nokta: (u_1, u_2). Mahkûmun İkilemi'nin dört sonucu (S: Sessiz, İ: İhanet), kazanç = −(hapis yılı).

Adım 1 / 3

- **Pareto baskınlık:** x, y'yi baskılar ⇔ herkes için u(x) ≥ u(y) ve en az biri için >.
- **Pareto optimal:** hiçbir sonuç onu baskılamaz; kimseyi kötüleştirmeden kimse iyileştirilemez.
- Grafikte: noktanın sağ üst çeyreğinde başka sonuç yoksa Pareto optimaldir.
- Pareto optimallik **adalet değildir**: (S, İ) de Pareto optimaldir ama biri 3 yıl yatar.

Denge (kararlılık) ile verimlilik ayrı sorulardır; aradaki kayıp Anarşinin Bedeli ile ölçülür.

Pareto Optimallik: Kazanç Uzayında Bakış

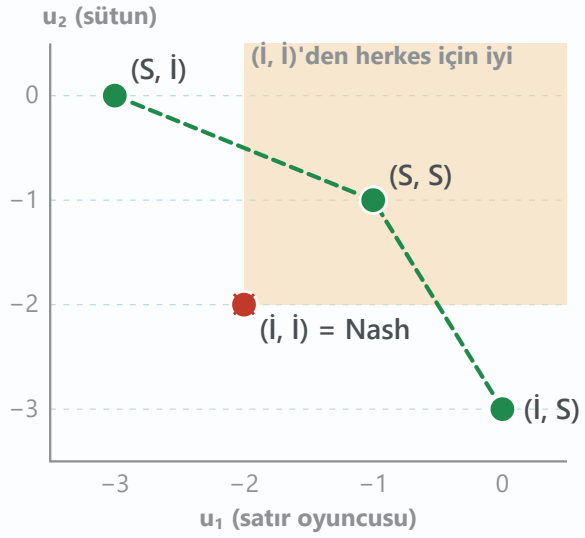


Nash dengesi (İ, İ) = (−2, −2). Turuncu bölgede ikisine de daha iyi bir sonuç var: **(S, S)**.

- **Pareto baskınlık:** x, y'yi baskılar ⇔ herkes için u(x) ≥ u(y) ve en az biri için >.
- **Pareto optimal:** hiçbir sonuç onu baskılamaz; kimseyi kötüleştirmeden kimse iyileştirilemez.
- Grafikte: noktanın sağ üst çeyreğinde başka sonuç yoksa Pareto optimaldir.
- Pareto optimallik **adalet değildir**: (S, İ) de Pareto optimaldir ama biri 3 yıl yatar.

Denge (kararlılık) ile verimlilik ayrı sorulardır; aradaki kayıp Anarşinin Bedeli ile ölçülür.

Pareto Optimallik: Kazanç Uzayında Bakış



Pareto optimal: (S, S), (S, İ), (İ, S) (yeşil). Tek Nash dengesi olan (İ, İ) ise Pareto baskılanır.

- **Pareto baskınlık:** x, y'yi baskılar ⇔ herkes için u(x) ≥ u(y) ve en az biri için >.
- **Pareto optimal:** hiçbir sonuç onu baskılamaz; kimseyi kötüleştirmeden kimse iyileştirilemez.
- Grafikte: noktanın sağ üst çeyreğinde başka sonuç yoksa Pareto optimaldir.
- Pareto optimallik **adalet değildir**: (S, İ) de Pareto optimaldir ama biri 3 yıl yatar.

Denge (kararlılık) ile verimlilik ayrı sorulardır; aradaki kayıp Anarşinin Bedeli ile ölçülür.

Oyunların Sınıflandırılması

Boyut	Bir uç	Diğer uç	Sorulan soru	Örnek
Zamanlama	Eşzamanlı	Sıralı	Rakibin hamlesini görerek mi oynuyorum?	kapalı zarf teklif / piyasaya giriş
Anlaşma	İşbirlikçi	İşbirlikçi olmayan	Bağlayıcı anlaşma yapılabilir mi?	ortak girişim / Mahkûmun İkilemi
Toplam kazanç	Sıfır toplamlı	Sıfır toplamlı olmayan	Birinin kazancı diğerinin kaybı mı?	penaltı / iklim anlaşması
Simetri	Simetrik	Asimetrik	Oyuncuların yerini değiştirince oyun aynı mı?	Geyik Avı / Cinsiyetler Savaşı
Bilgi	Mükemmel bilgi	Mükemmel olmayan bilgi	Geçmiş hamlelerin hepsini görüyor muyum?	Connect Four / poker
Ufuk	Sonlu	Sonsuz	Oyunun belirli bir sonu var mı?	Kırkayak / sürekli fiyat rekabeti

Boyutlar birbirinden bağımsızdır: satranç sıralı, işbirlikçi olmayan, sıfır toplamlı, mükemmel bilgili ve sonlu bir oyundur. Eşzamanlı oyun matrisle, sıralı oyun ağaçla gösterilir; sıralı oyun geriye doğru çıkarımla çözülür.

Simetrik ve Asimetrik Oyunlar

Simetrik

	G	T
G	2, 2	0, 1
T	1, 0	1, 1

$u_1(G,T) = u_2(T,G) = 0$

- Oyuncuların strateji kümeleri aynı ve **$u_1(a, b) = u_2(b, a)$**: kazanç yalnızca stratejiye bağlı, kimin oynadığına değil.
- Matris "köşegene göre aynalı" görünür.

`Mahkûmun İkilemi, Geyik Avı, Tavuk`

Asimetrik

	O	F
O	3, 2	0, 0
F	0, 0	2, 3

$u_1(O,O) = 3 \neq u_2(O,O) = 2$

- Oyuncuların rolleri, stratejileri ya da kazançları farklıdır.
- Aynı sonuç oyunculara farklı değer verir.

`Cinsiyetler Savaşı, piyasaya giriş, Yazı–Tura`

Sonlu ve Sonsuz Ufuklu Oyunlar

Sonlu ufuk



- Oyun önceden belirli sayıda dönem ya da hamleden sonra biter.
- Son dönemin davranışı önceki kararları etkiler; **geriye doğru çıkarım** kullanılır.

`Tek seferlik pazarlık, T turlu tekrarlanan oyun`

Sonsuz ufuk



- Oyun belirsiz süre sürebilir; gelecek kazançlar **iskonto faktörü** $\delta \in (0, 1)$ ile tartılır: $\Sigma\, \delta^t u_t$.
- Gelecek yeterince önemliyse tehdit, ödül ve itibar işbirliğini destekler.

`Sürekli fiyat rekabeti, ortak kaynak kullanımı`

"Sonlu oyun" terimi ayrıca oyuncu ve strateji kümeleri sonlu olan stratejik oyunlar için de kullanılır (Nash teoremi bu anlamdadır).

Yazı-Tura Eşleştirme (Matching Pennies)

- İki oyuncu aynı anda bir madeni parayı **Yazı** ya da **Tura** yüzüyle gösterir.
- Paralar **eşleşirse** satır oyuncusu (Eşleştiren), **eşleşmezse** sütun oyuncusu (Ayrıştıran) 1 birim kazanır.
- Gerçek hayatta: penaltıda kaleci–atıcı, güvenlik denetiminde denetçi–kaçakçı.

satır ╲ sütun	Yazı	Tura
Yazı	(1, −1)	(−1, 1)
Tura	(−1, 1)	(1, −1)

Özellikler

- Sıfır toplamlı ($u_1 + u_2 = 0$), asimetrik.
- Çıkarlar tamamen zıt: saf dengesi yok.

ÇATIŞMA OYUNLARI

Yazı-Tura Eşleştirme: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Yazı	Tura
Yazı	(1 , −1)	(−1 , 1)
Tura	(−1 , 1)	(1 , −1)

Satır oyuncusunun her sütuna, sütun oyuncusunun her satıra **en iyi tepkisini** (best response) işaretleyeceğiz. İki işaretli hücre = saf Nash dengesi.

Adım 1 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Yazı-Tura Eşleştirme: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Yazı	Tura
Yazı	(1 , −1)	(−1 , 1)
Tura	(−1 , 1)	(1 , −1)

Sütun oyuncusu **Yazı** oynarsa satır oyuncusu: Yazı → 1, Tura → −1 ⇒ en iyi tepki **Yazı**.

Adım 2 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Yazı-Tura Eşleştirme: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Yazı	Tura
Yazı	(**1** , −1)	(−1 , 1)
Tura	(−1 , 1)	(**1** , −1)

Sütun oyuncusu **Tura** oynarsa satır oyuncusu: Yazı → −1, Tura → 1 ⇒ en iyi tepki **Tura**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Yazı-Tura Eşleştirme: Saf Dengeleri İşaretle

	sütun oyuncusu: Yazı	sütun oyuncusu: Tura
satır ↓ Yazı	(**1** , −1)	(−1 , **1**)
Tura	(−1 , 1)	(**1** , −1)

Satır oyuncusu **Yazı** oynarsa sütun oyuncusu: Yazı → −1, Tura → 1 ⇒ en iyi tepki **Tura**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Yazı-Tura Eşleştirme: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Yazı	Tura
Yazı	(**1** , −1)	(−1 , **1**)
Tura	(−1 , **1**)	(**1** , −1)

Satır oyuncusu **Tura** oynarsa sütun oyuncusu: Yazı → 1, Tura → −1 ⇒ en iyi tepki **Yazı**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Yazı-Tura Eşleştirme: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Yazı	Tura
Yazı	(**1** , −1)	(−1 , **1**)
Tura	(−1 , **1**)	(**1** , −1)

Hiçbir hücrede iki işaret yok ⇒ **saf Nash dengesi yok**. Her sonuçta oyunculardan biri sapmak ister (işaretler döngü oluşturur).

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Yazı-Tura Eşleştirme: Sonuçların Yorumu

kim sapar?	Yazı	Tura
Yazı	(1, −1) sütun → Tura	(−1, 1) satır → Tura
Tura	(−1, 1) satır → Yazı	(1, −1) sütun → Yazı

Her hücrede kaybeden taraf sapar: sapmalar bir döngü kurar

Saf denge yok

Çıkarlar tamamen zıt: bir oyuncunun memnun olduğu her hücrede öteki sapmak ister.

Neden rastgele?

Önceden kestirilebilen her eğilim sömürülür: sütun %60 Yazı derse satır hep Yazı der ve kazanır.

Gerçek hayatta

Penaltıda atıcı da kaleci de tahmin edilemez olmaya çalışır; denetçi de denetim gününü gizler.

Kararlı bir saf sonuç olmadığında tek çıkış rastgele oynamaktır (**karışık strateji**): amaç rakibe bilgi vermemek. Rastgelelik oyuncuyu sömürülemez kılar, ama kazanç garantisi vermez.

Tavuk (Chicken)

- İki sürücü dar bir yolda birbirine doğru ilerler. İlk **kıran** "tavuk" (korkak) olur (−1), düz giden kazanır (+1).
- İkisi de düz giderse **çarpışırlar** (−10, −10); ikisi de kırarsa berabere (0, 0).
- Kavşakta geçiş hakkı, fiyat savaşları, caydırma krizleri bu yapıdadır (Şahin–Güvercin oyunu da denktir).

satır ╲ sütun	Kır	Düz git
Kır	(0, 0)	(−1, 1)
Düz git	(1, −1)	(−10, −10)

Özellikler

- Simetrik, sıfır toplamlı değil.
- **Anti-koordinasyon**: farklı eylem seçmek istenir; bir **cesaret (caydırma) ikilemi**.

Tavuk: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Kır	Düz git
Kır	(0 , 0)	(−1 , 1)
Düz git	(1 , −1)	(−10 , −10)

Satır oyuncusunun her sütuna, sütun oyuncusunun her satıra **en iyi tepkisini** (best response) işaretleyeceğiz. İki işaretli hücre = saf Nash dengesi.

Adım 1 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Tavuk: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Kır	Düz git
Kır	(0 , 0)	(−1 , 1)
Düz git	(1 , −1)	(−10 , −10)

Sütun oyuncusu **Kır** oynarsa satır oyuncusu: Kır → 0, Düz git → 1 ⇒ en iyi tepki **Düz git**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Tavuk: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Kır	Düz git
Kır	(0 , 0)	(−1 , 1)
Düz git	(1 , −1)	(−10 , −10)

Sütun oyuncusu **Düz git** oynarsa satır oyuncusu: Kır → −1, Düz git → −10 ⇒ en iyi tepki **Kır**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Tavuk: Saf Dengeleri İşaretle

satır ↓	sütun oyuncusu: Kır	sütun oyuncusu: Düz git
Kır	(0 , 0)	(−1 , 1)
Düz git	(1 , −1)	(−10 , −10)

Satır oyuncusu **Kır** oynarsa sütun oyuncusu: Kır → 0, Düz git → 1 ⇒ en iyi tepki **Düz git**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Tavuk: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Kır	Düz git
Kır	(0 , 0)	(−1 , 1)
Düz git	(1 , −1)	(−10 , −10)

Satır oyuncusu **Düz git** oynarsa sütun oyuncusu: Kır → −1, Düz git → −10 ⇒ en iyi tepki **Kır**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Tavuk: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Kır	Düz git
Kır	(0 , 0)	(−1 , 1)
Düz git	(1 , −1)	(−10 , −10)

İki işaretli hücre(ler): **(Kır, Düz git), (Düz git, Kır)** ⇒ 2 saf Nash dengesi. Hiçbir oyuncu tek başına sapmakla kazanamaz.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Tavuk: Sonuçların Yorumu

satır ╲ sütun	Kır	Düz git
Kır	(0, 0)	(−1, 1)
Düz git	(1, −1)	(−10, −10)

Çerçeve: saf Nash dengesi · turuncu: çarpışma

Kim kıracak?

İki saf denge var ve ikisi de bir sürücüyü kayırır: kıran −1, düz giden +1 alır. Kuram hangisinin oynanacağını söylemez.

Anti-koordinasyon

Koordinasyon oyunlarının tersine oyuncular farklı eylem seçmek ister; aynı eylem ya boşa (0, 0) ya da felakete (−10, −10) gider.

Taahhüt

Geri adım atamayacağını inandırıcı biçimde gösteren (direksiyonu kilitleyen) sürücü rakibini kırmaya zorlar.

En kötü sonuç iki tarafın da geri adım atmamasıdır; kavşakta geçiş hakkı ve caydırma krizleri bu yüzden kurallara, işaretlere ve itibara dayanır.

Cinsiyetler Savaşı (Battle of the Sexes)

- Bir çift akşam buluşacak; iletişim kuramıyorlar.
- **Satır oyuncusu operayı**, **sütun oyuncusu futbolu** tercih ediyor; ama ikisi için de ayrı yerlere gitmek en kötüsü (0, 0).
- Standart savaşları (ör. iki firmanın farklı format desteklemesi) aynı yapıdadır.

satır ╲ sütun	Opera	Futbol
Opera	(3, 2)	(0, 0)
Futbol	(0, 0)	(2, 3)

Özellikler

- Asimetrik, sıfır toplamlı değil.
- Koordinasyon + çatışma: buluşmak ortak çıkar, *nerede* buluşulacağı çatışma.

KOORDİNASYON OYUNLARI

Cinsiyetler Savaşı: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Opera	Futbol
Opera	(3 , 2)	(0 , 0)
Futbol	(0 , 0)	(2 , 3)

Satır oyuncusunun her sütuna, sütun oyuncusunun her satıra **en iyi tepkisini** (best response) işaretleyeceğiz. İki işaretli hücre = saf Nash dengesi.

Adım 1 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Cinsiyetler Savaşı: Saf Dengeleri İşaretle

	sütun oyuncusu	
satır ↓	**Opera**	**Futbol**
Opera	(3 , 2)	(0 , 0)
Futbol	(0 , 0)	(2 , 3)

Sütun oyuncusu **Opera** oynarsa satır oyuncusu: Opera → 3, Futbol → 0 ⇒ en iyi tepki **Opera**.

Adım 2 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Cinsiyetler Savaşı: Saf Dengeleri İşaretle

satır ↓	sütun oyuncusu: Opera	sütun oyuncusu: Futbol
Opera	(3 , 2)	(0 , 0)
Futbol	(0 , 0)	(2 , 3)

Sütun oyuncusu **Futbol** oynarsa satır oyuncusu: Opera → 0, Futbol → 2 ⇒ en iyi tepki **Futbol**.

Adım 3 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Cinsiyetler Savaşı: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Opera	Futbol
Opera	(**3** , **2**)	(0 , 0)
Futbol	(0 , 0)	(**2** , 3)

Satır oyuncusu **Opera** oynarsa sütun oyuncusu: Opera → 2, Futbol → 0 ⇒ en iyi tepki **Opera**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Cinsiyetler Savaşı: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Opera	Futbol
Opera	(**3** , **2**)	(0 , 0)
Futbol	(0 , 0)	(**2** , **3**)

Satır oyuncusu **Futbol** oynarsa sütun oyuncusu: Opera → 0, Futbol → 3 ⇒ en iyi tepki **Futbol**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Cinsiyetler Savaşı: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Opera	Futbol
Opera	(3, 2)	(0, 0)
Futbol	(0, 0)	(2, 3)

İki işaretli hücre(ler): **(Opera, Opera), (Futbol, Futbol)** ⇒ 2 saf Nash dengesi. Hiçbir oyuncu tek başına sapmakla kazanamaz.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Cinsiyetler Savaşı: Sonuçların Yorumu

satır ╲ sütun	Opera	Futbol
Opera	(3, 2)	(0, 0)
Futbol	(0, 0)	(2, 3)

Çerçeve: saf Nash dengesi · turuncu: buluşamama

Kim kayırılır?

(Opera, Opera) satır oyuncusuna 3, (Futbol, Futbol) sütun oyuncusuna 3 verir: iki denge iki farklı oyuncuyu kayırır.

Koordinasyon + çatışma

Buluşmak ikisinin de çıkarına (ayrı yerler 0); nerede buluşulacağı ise çatışma konusudur.

Baskın strateji yok

En iyi seçim karşı tarafın ne yapacağına bağlıdır; Mahkûmun İkilemi'ndeki gibi tek doğru cevap yoktur.

Kuram iki dengeden hangisinin oynanacağını söylemez; rastgele seçmek de sık sık buluşamamaya yol açar. Çözüm oyunun dışından gelir: iletişim, gelenek, sırayla seçmek; standart savaşlarında ise pazara önce çıkan firma.

Geyik Avı (Stag Hunt)

- İki avcı: **geyik** ancak ikisi birlikte avlanırsa yakalanır (2, 2); **tavşanı** tek başına kesin yakalarsın (1).
- Tek başına geyik peşine düşen eli boş kalır (0).
- Rousseau'nun örneği; ortak projeler, iklim anlaşmaları, yeni teknolojiye birlikte geçiş.

satır ╲ sütun	Geyik	Tavşan
Geyik	(2, 2)	(0, 1)
Tavşan	(1, 0)	(1, 1)

Özellikler

- Simetrik, sıfır toplamlı değil.
- (Geyik, Geyik) Pareto optimal; (Tavşan, Tavşan) güvenli.

KOORDİNASYON OYUNLARI

Geyik Avı: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Geyik	Tavşan
Geyik	(2 , 2)	(0 , 1)
Tavşan	(1 , 0)	(1 , 1)

Satır oyuncusunun her sütuna, sütun oyuncusunun her satıra **en iyi tepkisini** (best response) işaretleyeceğiz. İki işaretli hücre = saf Nash dengesi.

Adım 1 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Geyik Avı: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Geyik	Tavşan
Geyik	(2 , 2)	(0 , 1)
Tavşan	(1 , 0)	(1 , 1)

Sütun oyuncusu **Geyik** oynarsa satır oyuncusu: Geyik → 2, Tavşan → 1 ⇒ en iyi tepki **Geyik**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Geyik Avı: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Geyik	Tavşan
Geyik	(2 , 2)	(0 , 1)
Tavşan	(1 , 0)	(1 , 1)

Sütun oyuncusu **Tavşan** oynarsa satır oyuncusu: Geyik → 0, Tavşan → 1 ⇒ en iyi tepki **Tavşan**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Geyik Avı: Saf Dengeleri İşaretle

satır ↓	sütun oyuncusu: Geyik	sütun oyuncusu: Tavşan
Geyik	(**2** , **2**)	(0 , 1)
Tavşan	(1 , 0)	(**1** , 1)

Satır oyuncusu **Geyik** oynarsa sütun oyuncusu: Geyik → 2, Tavşan → 1 ⇒ en iyi tepki **Geyik**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Geyik Avı: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Geyik	Tavşan
Geyik	(**2** , **2**)	(0 , 1)
Tavşan	(1 , 0)	(**1** , **1**)

Satır oyuncusu **Tavşan** oynarsa sütun oyuncusu: Geyik → 0, Tavşan → 1 ⇒ en iyi tepki **Tavşan**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Geyik Avı: Saf Dengeleri İşaretle

satır ↓	sütun oyuncusu: Geyik	sütun oyuncusu: Tavşan
Geyik	(2, 2)	(0, 1)
Tavşan	(1, 0)	(1, 1)

İki işaretli hücre(ler): **(Geyik, Geyik), (Tavşan, Tavşan)** ⇒ 2 saf Nash dengesi. Hiçbir oyuncu tek başına sapmakla kazanamaz.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

KOORDİNASYON OYUNLARI

Geyik Avı: Sonuçların Yorumu

satır ╲ sütun	Geyik	Tavşan
Geyik	(2, 2)	(0, 1)
Tavşan	(1, 0)	(1, 1)

Çerçeve: saf Nash dengesi · turuncu: avcılar ayrı düşer

İki saf denge

(Geyik, Geyik) = (2, 2) Pareto optimal; (Tavşan, Tavşan) = (1, 1) daha kötü ama güvenli.

Risk

Geyiğe gidip yalnız kalan 0 alır; tavşan, karşı taraf ne yaparsa yapsın 1 getirir.

Güven

Karşı tarafın geyiğe geleceğinden yeterince emin olan geyiğe gider; emin olmayan için tavşan en iyi tepkidir.

Bu bir **güven ikilemi**dir: (Geyik, Geyik) herkes için en iyisi ve bir denge, ama karşı tarafa güvenmeyen için Tavşan risksizdir. Mahkûmun İkilemi'nden farkı: burada işbirliği **dengedir**; sorun çıkar çatışması değil, güven eksikliğidir.

Saf Koordinasyon (Pure Coordination)

- İki arkadaş akşam **partiye** mi gidecek, **evde** mi kalacak? İletişim yok.
- İkisi de aynı şeyi yapmak ister; birlikte parti (10, 10) birlikte evden (5, 5) daha iyi.
- Farklı karar verirlerse ikisi de 0 alır: evde kalan da parti kaçıran da yalnız kalır.

satır ╲ sütun	Parti	Ev
Parti	(10, 10)	(0, 0)
Ev	(0, 0)	(5, 5)

Özellikler

- Simetrik; **ortak çıkar**: her hücrede $u_1 = u_2$.
- İki saf denge Pareto sıralı: (Parti, Parti) daha iyi.

KOORDİNASYON OYUNLARI

Saf Koordinasyon: Saf Dengeleri İşaretle

		sütun oyuncusu: Parti	sütun oyuncusu: Ev
satır ↓			
Parti		(10 , 10)	(0 , 0)
Ev		(0 , 0)	(5 , 5)

Satır oyuncusunun her sütuna, sütun oyuncusunun her satıra **en iyi tepkisini** (best response) işaretleyeceğiz. İki işaretli hücre = saf Nash dengesi.

Adım 1 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Saf Koordinasyon: Saf Dengeleri İşaretle

satır ↓	Parti (sütun oyuncusu)	Ev (sütun oyuncusu)
Parti	(10 , 10)	(0 , 0)
Ev	(0 , 0)	(5 , 5)

Sütun oyuncusu **Parti** oynarsa satır oyuncusu: Parti → 10, Ev → 0 ⇒ en iyi tepki **Parti**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Saf Koordinasyon: Saf Dengeleri İşaretle

satır ↓	Parti (sütun oyuncusu)	Ev (sütun oyuncusu)
Parti	(**10** , 10)	(0 , 0)
Ev	(0 , 0)	(**5** , 5)

Sütun oyuncusu **Ev** oynarsa satır oyuncusu: Parti → 0, Ev → 5 ⇒ en iyi tepki **Ev**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Saf Koordinasyon: Saf Dengeleri İşaretle

satır ↓	Parti (sütun oyuncusu)	Ev (sütun oyuncusu)
Parti	(**10** , **10**)	(0 , 0)
Ev	(0 , 0)	(**5** , 5)

Satır oyuncusu **Parti** oynarsa sütun oyuncusu: Parti → 10, Ev → 0 ⇒ en iyi tepki **Parti**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Saf Koordinasyon: Saf Dengeleri İşaretle

satır ↓	Parti (sütun oyuncusu)	Ev (sütun oyuncusu)
Parti	(**10** , **10**)	(0 , 0)
Ev	(0 , 0)	(**5** , **5**)

Satır oyuncusu **Ev** oynarsa sütun oyuncusu: Parti → 0, Ev → 5 ⇒ en iyi tepki **Ev**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Saf Koordinasyon: Saf Dengeleri İşaretle

satır ↓	Parti	Ev
	sütun oyuncusu	
Parti	(10, 10)	(0, 0)
Ev	(0, 0)	(5, 5)

İki işaretli hücre(ler): **(Parti, Parti), (Ev, Ev)** ⇒ 2 saf Nash dengesi. Hiçbir oyuncu tek başına sapmakla kazanamaz.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Saf Koordinasyon: Sonuçların Yorumu

satır ╲ sütun	Parti	Ev
Parti	(10, 10)	(0, 0)
Ev	(0, 0)	(5, 5)

Çerçeve: saf Nash dengesi · turuncu: farklı karar

Pareto sıralı iki denge

(Parti, Parti) = (10, 10), (Ev, Ev) = (5, 5) sonucundan ikisi için de iyidir; ama ikisi de dengedir.

Çıkar çatışması yok

Her hücrede $u_1 = u_2$; sorun yalnızca beklentileri buluşturmaktır.

Evde kalmak "garantili" değil

Karşı taraf partiye giderse evde kalan 0 alır. Güvenli strateji yoktur.

Kötü denge (Ev, Ev) de kendi kendini sürdürür: herkes ötekinin evde kalacağını beklerse kimse tek başına partiye gitmez. Gerçekte oyuncular bir **odak noktasına** (Schelling) yönelir: (Parti, Parti) hem Pareto hem de belirgin olduğu için seçilir.

Bedava Para (Free Money)

- Her iki oyuncu **Yazı** derse ikisine de 1 TL verilir; diğer üç sonuçta kimse bir şey kazanmaz.
- Masada "bedava para" var; tek gereken ikisinin de Yazı demesi.
- Basit görünür ama iki farklı denge türünü ayırt etmeyi öğretir.

satır ╲ sütun	Yazı	Tura
Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

Özellikler

- Simetrik, ortak çıkar.
- Yazı **zayıf baskın** strateji: hiç kötü değil, bazen daha iyi.

KOORDİNASYON OYUNLARI

Bedava Para: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Yazı	Tura
Yazı	(1 , 1)	(0 , 0)
Tura	(0 , 0)	(0 , 0)

Satır oyuncusunun her sütuna, sütun oyuncusunun her satıra **en iyi tepkisini** (best response) işaretleyeceğiz. İki işaretli hücre = saf Nash dengesi.

Adım 1 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Bedava Para: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Yazı	Tura
Yazı	(1 , 1)	(0 , 0)
Tura	(0 , 0)	(0 , 0)

Sütun oyuncusu **Yazı** oynarsa satır oyuncusu: Yazı → 1, Tura → 0 ⇒ en iyi tepki **Yazı**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Bedava Para: Saf Dengeleri İşaretle

	sütun oyuncusu: Yazı	sütun oyuncusu: Tura
satır ↓ Yazı	(1, 1)	(0, 0)
satır ↓ Tura	(0, 0)	(0, 0)

Sütun oyuncusu **Tura** oynarsa satır oyuncusu: Yazı → 0, Tura → 0 ⇒ eşitlik: **ikisi de** en iyi tepki, ikisini de işaretle.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Bedava Para: Saf Dengeleri İşaretle

satır ↓	Yazı	Tura
	sütun oyuncusu	
Yazı	(1 , 1)	(0 , 0)
Tura	(0 , 0)	(0 , 0)

Satır oyuncusu **Yazı** oynarsa sütun oyuncusu: Yazı → 1, Tura → 0 ⇒ en iyi tepki **Yazı**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Bedava Para: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Yazı	Tura
Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

Satır oyuncusu **Tura** oynarsa sütun oyuncusu: Yazı → 0, Tura → 0 ⇒ eşitlik: **ikisi de** en iyi tepki.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Bedava Para: Saf Dengeleri İşaretle

	sütun oyuncusu: Yazı	sütun oyuncusu: Tura
satır ↓ Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

İki işaretli hücre(ler): **(Yazı, Yazı), (Tura, Tura)** ⇒ 2 saf Nash dengesi. Hiçbir oyuncu tek başına sapmakla kazanamaz.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

KOORDİNASYON OYUNLARI

Bedava Para: Sonuçların Yorumu

satır ╲ sütun	Yazı	Tura
Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

Düz çerçeve: kesin denge · kesikli: zayıf denge

Kesin denge (strict): (Yazı, Yazı)

Sapan oyuncu kesin kaybeder (1 → 0).

Zayıf denge: (Tura, Tura)

Sapmak kazandırmaz ama kaybettirmez de (0 → 0); karşı taraf Tura derken Yazı demek boşunadır.

Zayıf baskın strateji

Yazı, Tura'yı zayıf baskılar: hiç kötü değil, karşı taraf Yazı derse daha iyi.

İki profil de Nash tanımını sağlar, ama yalnız (Yazı, Yazı) inandırıcıdır. Zayıf baskılanan stratejiyi (Tura) elemek mantıklı dengeyi seçer; yine de zayıf eleme her oyunda güvenli değildir, bazı dengeleri kaybettirebilir.

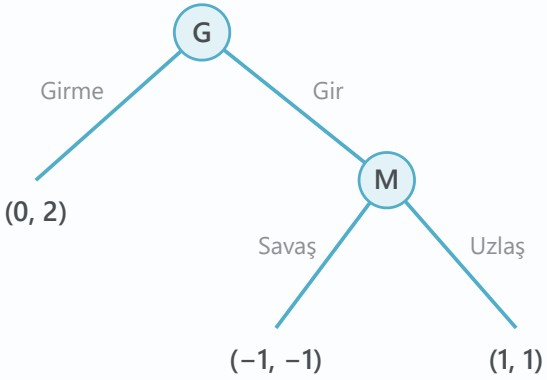
Alt Oyun ve Alt Oyun Mükemmel Denge

- **Alt oyun** (subgame): tek bir karar düğümünden başlayıp altındaki her şeyi içeren (bilgi kümelerini bölmeyen) oyun parçası.
- **Alt oyun mükemmel denge** (subgame perfect equilibrium, SPE): **her alt oyunda** Nash dengesi oluşturan strateji profili (Selten, 1965).
- Sıradan Nash dengesi, hiç ulaşılmayan düğümlerde **inandırıcı olmayan tehditlere** izin verebilir; SPE bunları eler.
- Sonlu ve mükemmel bilgili oyunlarda SPE **geriye doğru çıkarımla** (backward induction) bulunur (Kuhn/Zermelo).

Geriye doğru çıkarım

1. Yalnızca yapraklara bağlı son karar düğümlerini bul.
2. Orada karar veren oyuncunun en iyi dalını seç.
3. Düğümü, seçilen yaprağın kazancıyla değiştir.
4. Köke ulaşana kadar tekrarla.

Örnek: İnandırıcı Olmayan Tehdit



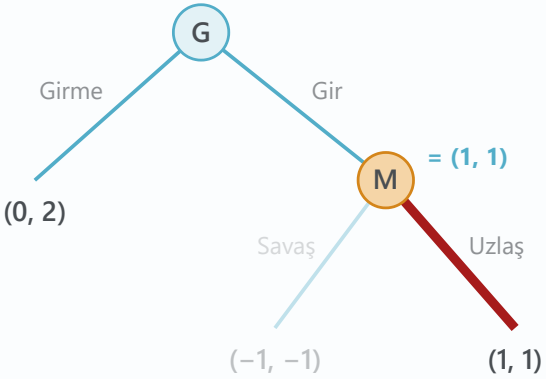
G önce seçer; M, yalnızca G girerse karar verir. Normal formda iki Nash dengesi vardır (sağdaki tablo).

Adım 1 / 3

	sütun oyuncusu: Savaş	sütun oyuncusu: Uzlaş
satır ↓ Gir	(−1 , −1)	(**1** , **1**)
Girme	(**0** , **2**)	(0 , **2**)

- (Gir, Uzlaş): SPE, geriye doğru çıkarımın sonucu.
- (Girme, Savaş): Nash dengesi, ama M'nin "girersen savaşırım" tehdidi **inandırıcı değil**: G girdiğinde savaşmak M'ye −1, uzlaşmak 1 verir.

Örnek: İnandırıcı Olmayan Tehdit



Son düğüm M: Savaş → −1, Uzlaş → 1 ⇒ **Uzlaş**. M'nin düğümünün değeri (1, 1).

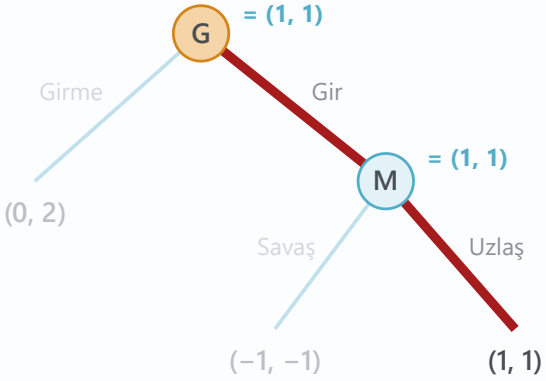
Adım 2 / 3

sütun oyuncusu

satır ↓	Savaş	Uzlaş
Gir	(−1 , −1)	(1 , 1)
Girme	(0 , 2)	(0 , 2)

- (Gir, Uzlaş): SPE, geriye doğru çıkarımın sonucu.
- (Girme, Savaş): Nash dengesi, ama M'nin "girersen savaşırım" tehdidi **inandırıcı değil**: G girdiğinde savaşmak M'ye −1, uzlaşmak 1 verir.

Örnek: İnandırıcı Olmayan Tehdit



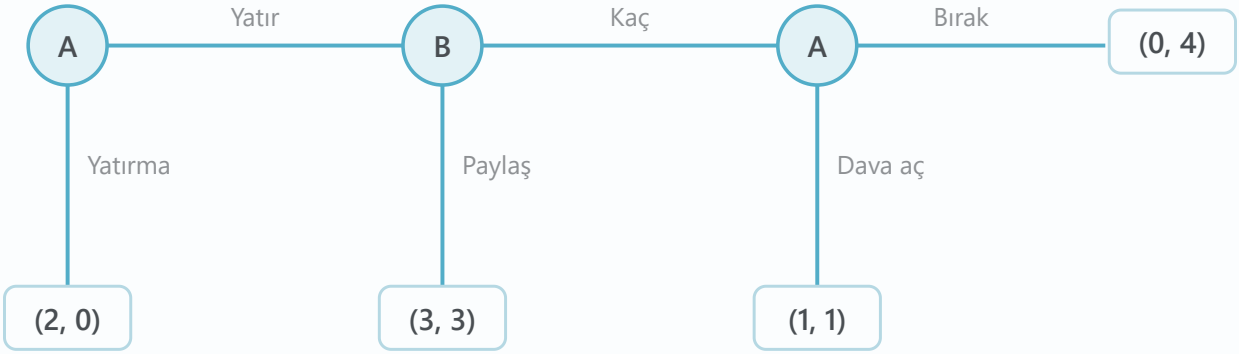
Kök G: Girme → 0, Gir → 1 (M'nin uzlaşacağını bilerek) ⇒ **Gir**. SPE: (Gir, Uzlaş), sonuç (1, 1).

sütun oyuncusu

satır ↓	Savaş	Uzlaş
Gir	(−1 , −1)	(1 , 1)
Girme	(0 , 2)	(0 , 2)

- (Gir, Uzlaş): SPE, geriye doğru çıkarımın sonucu.
- (Girme, Savaş): Nash dengesi, ama M'nin "girersen savaşırım" tehdidi **inandırıcı değil**: G girdiğinde savaşmak M'ye −1, uzlaşmak 1 verir.

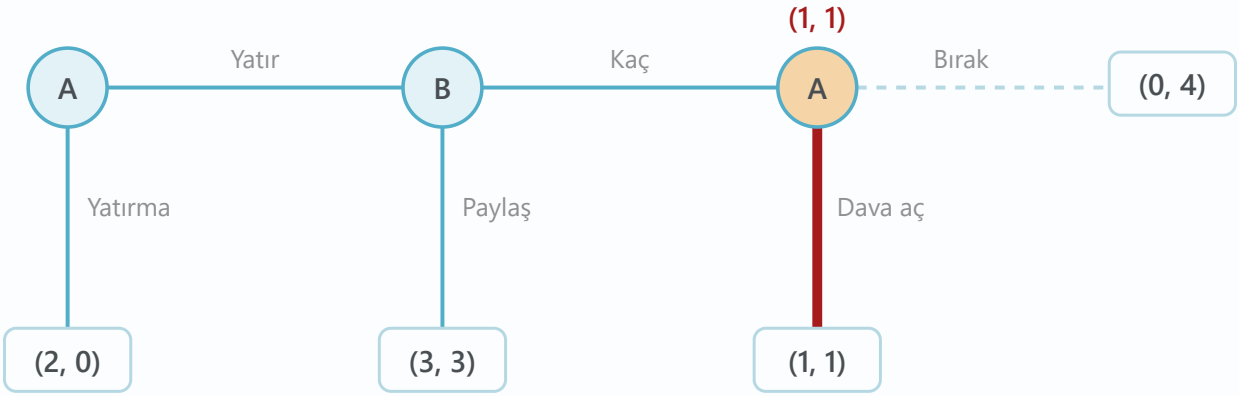
Adım Adım: Geriye Doğru Çıkarım (Yatırım Oyunu)



A: Ayşe (yatırımcı), B: Bora (girişimci); yaprakta (Ayşe, Bora)

Ayşe yatırım yapar ya da yapmaz. Yaparsa Bora kârı paylaşır ya da parayla kaçar. Kaçarsa Ayşe dava açar ya da bırakır. Çözüm **sondan** başlar.

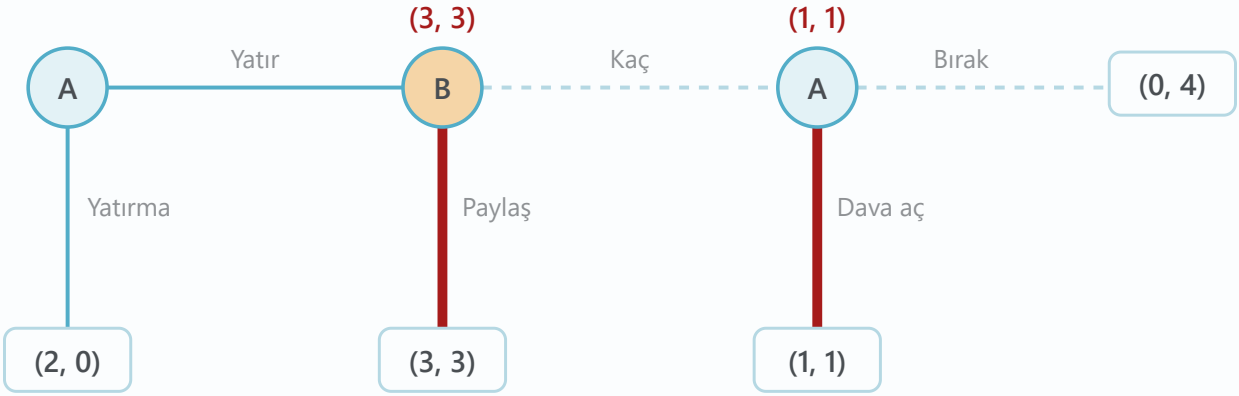
Adım Adım: Geriye Doğru Çıkarım (Yatırım Oyunu)



A: Ayşe (yatırımcı), B: Bora (girişimci); yaprakta (Ayşe, Bora)

Son karar düğümü (Ayşe, Bora kaçtıktan sonra): Dava aç 1 > Bırak 0 → **Dava aç**. Düğümün değeri (1, 1).

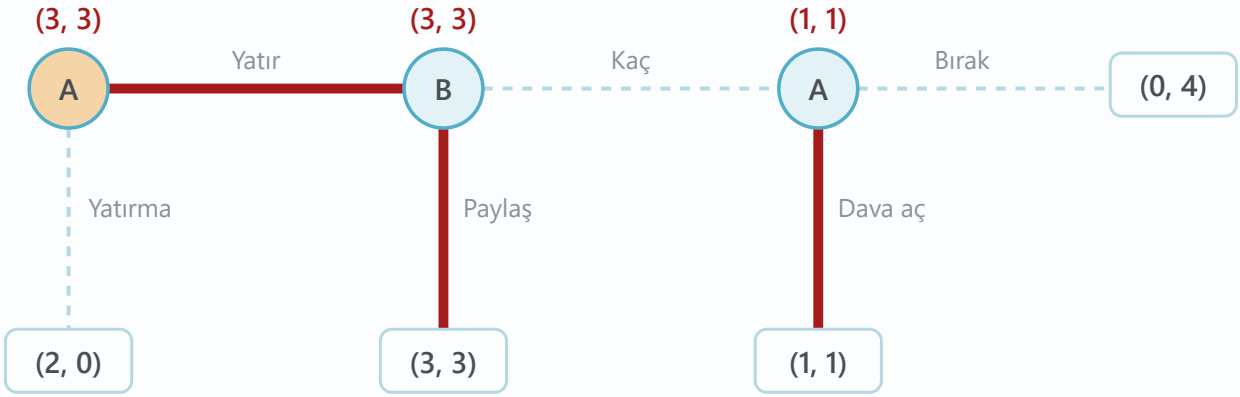
Adım Adım: Geriye Doğru Çıkarım (Yatırım Oyunu)



A: Ayşe (yatırımcı), B: Bora (girişimci); yaprakta (Ayşe, Bora)

Bora, kaçarsa dava yiyeceğini bilir: Paylaş 3 > Kaç 1 → **Paylaş**. Bora'nın düğümünün değeri (3, 3).

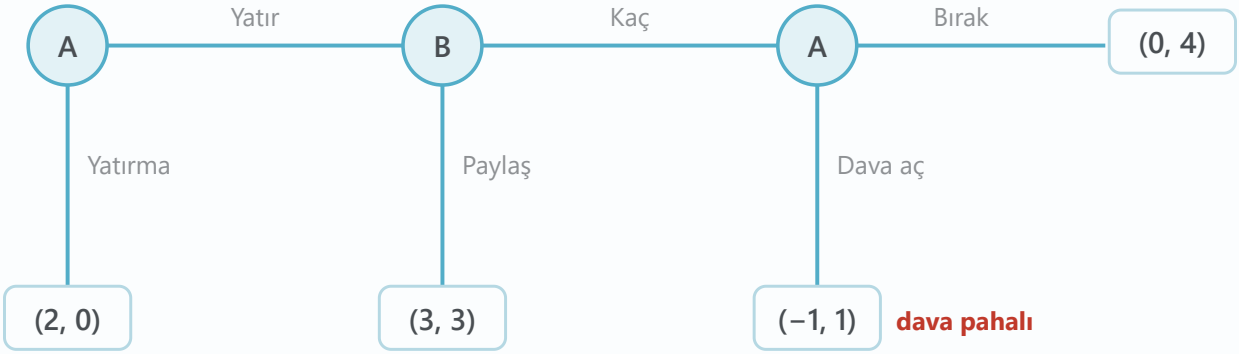
Adım Adım: Geriye Doğru Çıkarım (Yatırım Oyunu)



A: Ayşe (yatırımcı), B: Bora (girişimci); yaprakta (Ayşe, Bora)

Kök: Yatır 3 > Yatırma 2 → **Yatır**. Alt oyun mükemmel denge: Ayşe (Yatır, Dava aç), Bora Paylaş; sonuç **(3, 3)**.

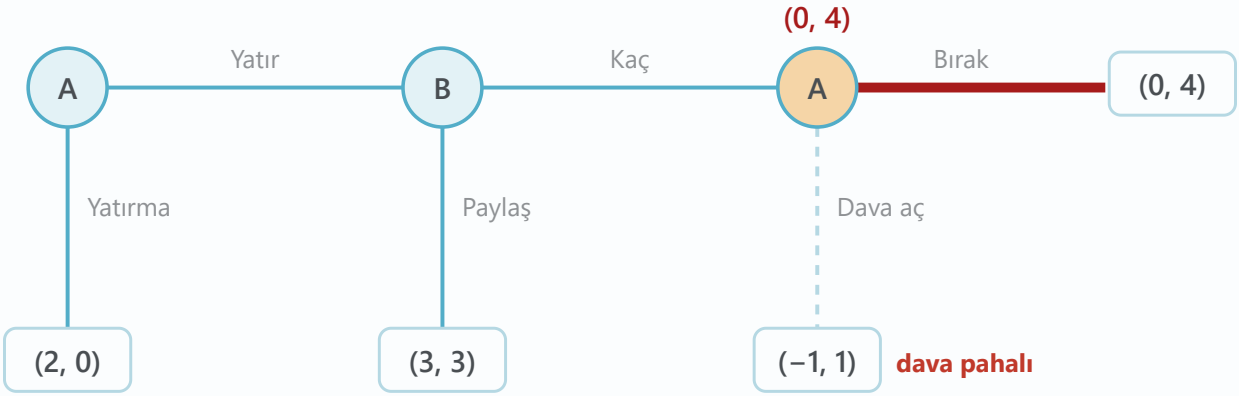
Adım Adım: Geriye Doğru Çıkarım (Yatırım Oyunu)



A: Ayşe (yatırımcı), B: Bora (girişimci); yaprakta (Ayşe, Bora)

Şimdi dava pahalı: dava açan Ayşe −1 alır. Ağaç aynı, yalnız bir yaprak değişti. Yeniden sondan başlanır.

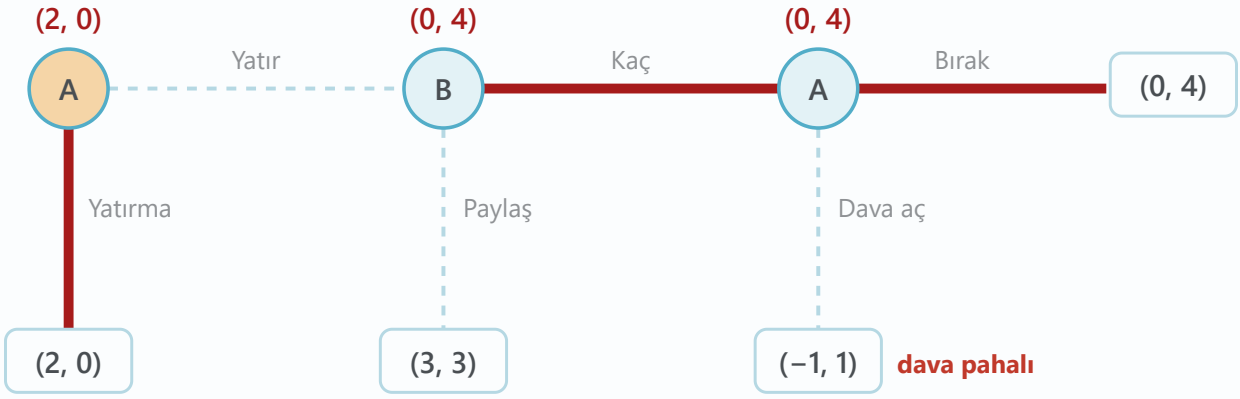
Adım Adım: Geriye Doğru Çıkarım (Yatırım Oyunu)



A: Ayşe (yatırımcı), B: Bora (girişimci); yaprakta (Ayşe, Bora)

Son düğüm: Bırak 0 > Dava aç −1 → **Bırak**. Düğümün değeri (0, 4).

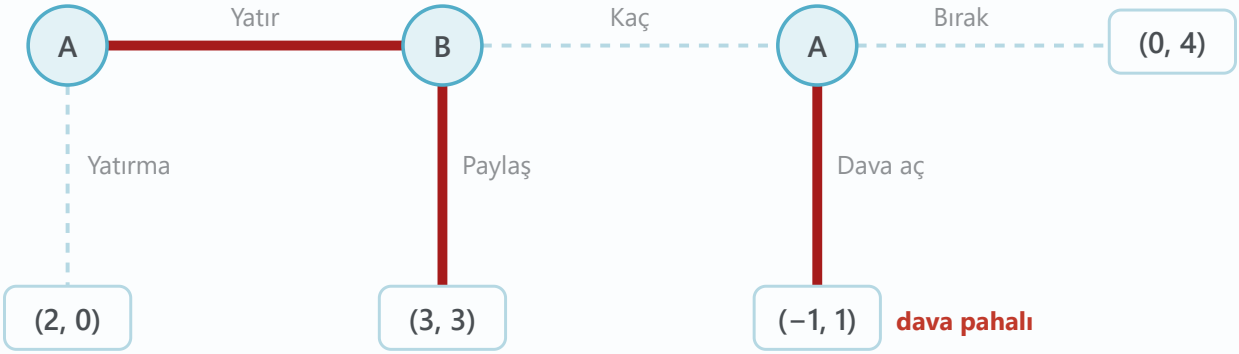
Adım Adım: Geriye Doğru Çıkarım (Yatırım Oyunu)



A: Ayşe (yatırımcı), B: Bora (girişimci); yaprakta (Ayşe, Bora)

Bora: Kaç 4 > Paylaş 3 → **Kaç**. Kök: Yatırma 2 > 0 → **Yatırma**. Sonuç (2, 0): ikisi de (3, 3)'ten kötü.

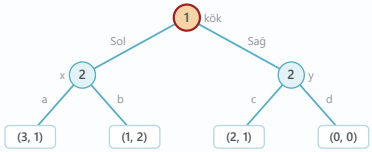
Adım Adım: Geriye Doğru Çıkarım (Yatırım Oyunu)



A: Ayşe (yatırımcı), B: Bora (girişimci); yaprakta (Ayşe, Bora)

"Kaçarsan dava açarım" planıyla (Yatır, Paylaş) yine bir **Nash dengesidir**: kimse tek başına sapmaz. Ama tehdit inandırıcı değil ($-1 < 0$): alt oyun mükemmel denge değil.

Kod İzleme: `cozum(d)`



Kökte oyuncu 1 (indis 0), x ve y'de oyuncu 2 (indis 1) seçer. cozum(kök) çağrılır.

Adım 1 / 10

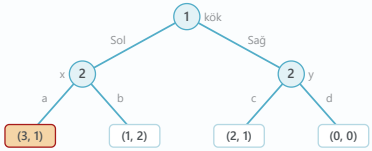
```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
▶      ad, oyuncu, dallar = d
▶      en_iyi = None
6      for hamle, cocuk in dallar:
7          v = cozum(cocuk)
8          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
9              en_iyi, secim = v, hamle
10     print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                  ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
▶  print(cozum(kok))
```

DEĞİŞKENLER

`ad = kök` `oyuncu = 0` `en_iyi = None`

ÇIKTI

Kod İzleme: `cozum(d)`



Sol dalı: cozum(x) → ilk dal a, yaprak (3, 1) olduğu gibi döner.

Adım 2 / 10

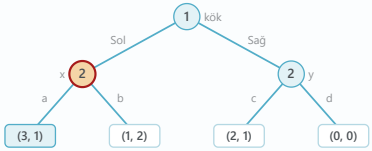
```
1  def cozum(d):
▶      if isinstance(d, tuple):          # yaprak: kazanç
▶          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
▶      for hamle, cocuk in dallar:
▶          v = cozum(cocuk)
8          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
9              en_iyi, secim = v, hamle
10     print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                  ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

DEĞİŞKENLER

`ad = x` `oyuncu = 1` `hamle = a` `v = (3, 1)`

ÇIKTI

Kod İzleme: `cozum(d)`



x'te en_iyi henüz yok: en_iyi = (3, 1), secim = a.

Adım 3 / 10

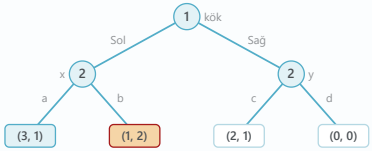
```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
6      for hamle, cocuk in dallar:
7          v = cozum(cocuk)
▶          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
▶              en_iyi, secim = v, hamle
10     print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                  ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

DEĞİŞKENLER

`ad = x` `oyuncu = 1` `v = (3, 1)` `en_iyi = (3, 1)` `secim = a`

ÇIKTI

Kod İzleme: `cozum(d)`



İkinci dal b: (1, 2). Oyuncu 2 kendi kazancına bakar: v[1] = 2 > en_iyi[1] = 1.

Adım 4 / 10

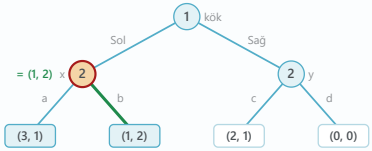
```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
▶      for hamle, cocuk in dallar:
▶          v = cozum(cocuk)
▶          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
9              en_iyi, secim = v, hamle
10     print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

DEĞİŞKENLER

`ad = x` `oyuncu = 1` `hamle = b` `v = (1, 2)` `en_iyi = (3, 1)`

ÇIKTI

Kod İzleme: `cozum(d)`



secim = b; “x b” yazılır, x'in değeri (1, 2). Oyuncu 1'in en sevdiği (3, 1) ulaşılamaz oldu.

Adım 5 / 10

```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
6      for hamle, cocuk in dallar:
7          v = cozum(cocuk)
8          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
▶              en_iyi, secim = v, hamle
▶      print(ad, secim)
▶      return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

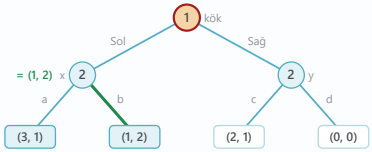
DEĞİŞKENLER

`ad = x` `en_iyi = (1, 2)` `secim = b`

ÇIKTI

```
x b
```

Kod İzleme: `cozum(d)`



Kökte Sol'un değeri (1, 2): en_iyi = (1, 2), secim = Sol.

Adım 6 / 10

```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
6      for hamle, cocuk in dallar:
▶          v = cozum(cocuk)
▶          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
▶              en_iyi, secim = v, hamle
10     print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

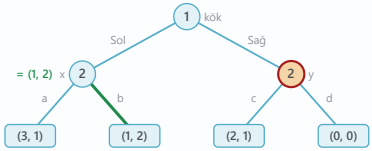
DEĞİŞKENLER

`ad = kök` `oyuncu = 0` `hamle = Sol` `v = (1, 2)` `en_iyi = (1, 2)` `secim = Sol`

ÇIKTI

```
x b
```

Kod İzleme: `cozum(d)`



Sağ dalı: cozum(y). c → (2, 1), d → (0, 0); oyuncu 2 için 1 > 0, secim = c.

Adım 7 / 10

```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
▶      for hamle, cocuk in dallar:
▶          v = cozum(cocuk)
▶          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
▶              en_iyi, secim = v, hamle
10     print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

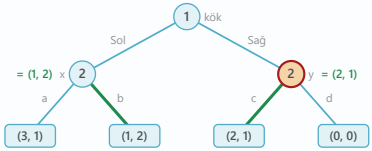
DEĞİŞKENLER

`ad = y` `oyuncu = 1` `en_iyi = (2, 1)` `secim = c`

ÇIKTI

```
x b
```

Kod İzleme: `cozum(d)`



“y c” yazılır; y'nin değeri (2, 1) döner.

Adım 8 / 10

```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
6      for hamle, cocuk in dallar:
7          v = cozum(cocuk)
8          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
9              en_iyi, secim = v, hamle
▶      print(ad, secim)
▶      return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

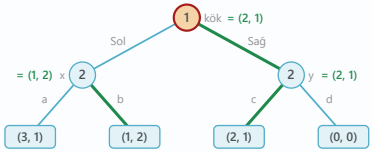
DEĞİŞKENLER

`ad = y` `en_iyi = (2, 1)` `secim = c`

ÇIKTI

```
x b
y c
```

Kod İzleme: `cozum(d)`



Kökte oyuncu 1 karşılaştırır: Sağ 2 > Sol 1 → secim = Sağ. “kök Sağ” yazılır.

Adım 9 / 10

```
 1 def cozum(d):
 2     if isinstance(d, tuple):          # yaprak: kazanç
 3         return d
 4     ad, oyuncu, dallar = d
 5     en_iyi = None
 6     for hamle, cocuk in dallar:
 7         v = cozum(cocuk)
 ▶           if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
 ▶               en_iyi, secim = v, hamle
 ▶       print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

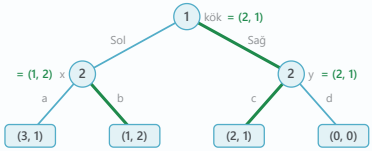
DEĞİŞKENLER

`ad = kök` `oyuncu = 0` `v = (2, 1)` `en_iyi = (2, 1)` `secim = Sağ`

ÇIKTI

```
x b
y c
kök Sağ
```

Kod İzleme: `cozum(d)`



Sonuç (2, 1). SPE: (Sağ; x'te b, y'de c). Ulaşılmayan x'teki b de stratejinin parçasıdır.

Adım 10 / 10

```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
6      for hamle, cocuk in dallar:
7          v = cozum(cocuk)
8          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
9              en_iyi, secim = v, hamle
10     print(ad, secim)
▶      return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
▶ print(cozum(kok))
```

DEĞİŞKENLER

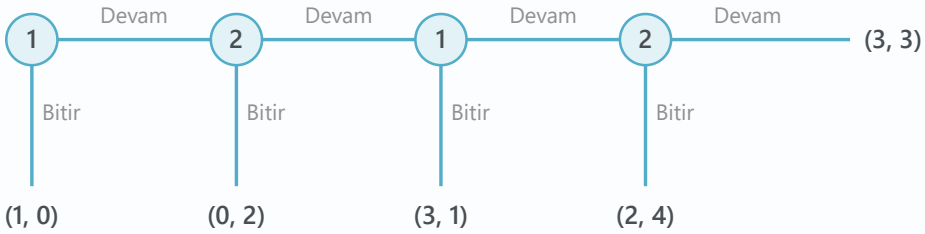
`sonuç = (2, 1)`

ÇIKTI

```
x b
y c
kök Sağ
(2, 1)
```

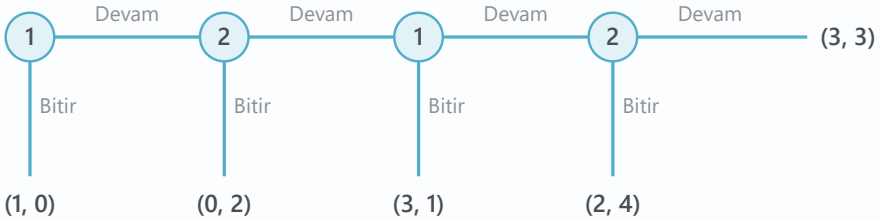
Kırkayak Oyunu (Centipede Game)

- İki oyuncu sırayla karar verir (Rosenthal, 1981). Her düğümde oyuncu ya **Bitir** der ve o anki paylaşım gerçekleşir, ya da **Devam** der ve toplam büyür.
- Toplam kazanç büyür (1 → 2 → 4 → 6; sonda da 6), ama devam eden oyuncu, rakibi bir sonraki adımda bitirirse daha az alır.



Düğümdeki sayı karar veren oyuncudur. Yapraklarda (Oyuncu 1, Oyuncu 2) kazancı; en sağda herkes devam ederse (3, 3).

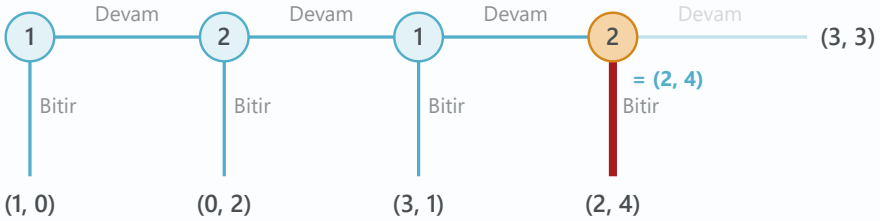
Kırkayak: Geriye Doğru Çıkarım Adım Adım



Düğüm	Karar veren	Bitir	Devamın değeri	Karar	Düğüm değeri
4	Oyuncu 2	(2, 4)	?	?	?
3	Oyuncu 1	(3, 1)	?	?	?
2	Oyuncu 2	(0, 2)	?	?	?
1	Oyuncu 1	(1, 0)	?	?	?

Oyuncu 1 başlar; her düğümde oyuncu **Bitir** (aşağı) ya da **Devam** (sağa) der. Son karar düğümünden (4. düğüm, Oyuncu 2) başlayıp geriye gideceğiz.

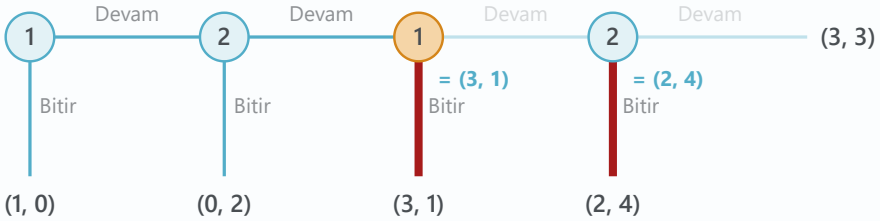
Kırkayak: Geriye Doğru Çıkarım Adım Adım



Düğüm	Karar veren	Bitir	Devamın değeri	Karar	Düğüm değeri
4	Oyuncu 2	(2, 4)	(3, 3)	**Bitir**	(2, 4)
3	Oyuncu 1	(3, 1)	?	?	?
2	Oyuncu 2	(0, 2)	?	?	?
1	Oyuncu 1	(1, 0)	?	?	?

4. düğüm, Oyuncu 2: Bitir → 4, Devam → 3 (devamın değeri (3, 3)). 4 > 3 ⇒ **Bitir**; düğümün değeri (2, 4).

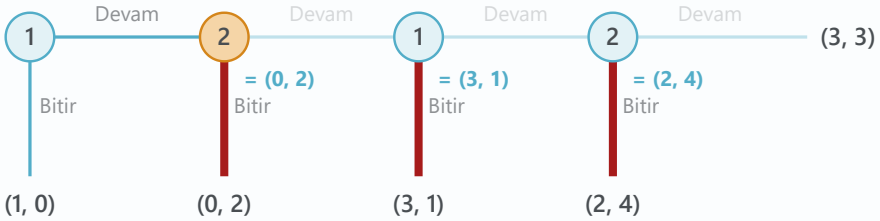
Kırkayak: Geriye Doğru Çıkarım Adım Adım



Düğüm	Karar veren	Bitir	Devamın değeri	Karar	Düğüm değeri
4	Oyuncu 2	(2, 4)	(3, 3)	**Bitir**	(2, 4)
3	Oyuncu 1	(3, 1)	(2, 4)	**Bitir**	(3, 1)
2	Oyuncu 2	(0, 2)	?	?	?
1	Oyuncu 1	(1, 0)	?	?	?

3. düğüm, Oyuncu 1: Bitir → 3, Devam → 2 (devamın değeri (2, 4)). 3 > 2 ⇒ **Bitir**; düğümün değeri (3, 1).

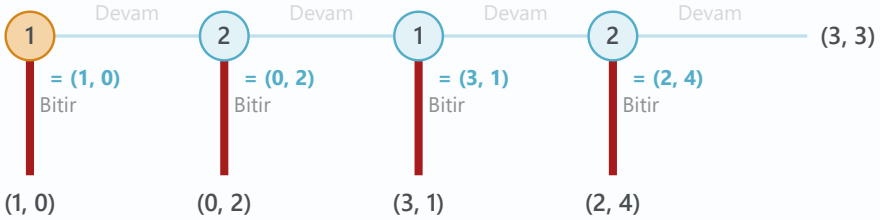
Kırkayak: Geriye Doğru Çıkarım Adım Adım



Düğüm	Karar veren	Bitir	Devamın değeri	Karar	Düğüm değeri
4	Oyuncu 2	(2, 4)	(3, 3)	**Bitir**	(2, 4)
3	Oyuncu 1	(3, 1)	(2, 4)	**Bitir**	(3, 1)
2	Oyuncu 2	(0, 2)	(3, 1)	**Bitir**	(0, 2)
1	Oyuncu 1	(1, 0)	?	?	?

2. düğüm, Oyuncu 2: Bitir → 2, Devam → 1 (devamın değeri (3, 1)). 2 > 1 ⇒ **Bitir**; düğümün değeri (0, 2).

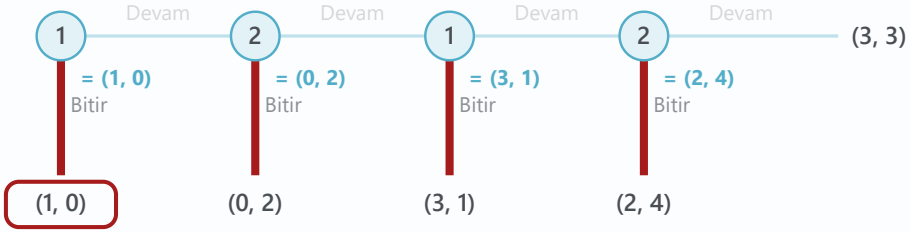
Kırkayak: Geriye Doğru Çıkarım Adım Adım



Düğüm	Karar veren	Bitir	Devamın değeri	Karar	Düğüm değeri
4	Oyuncu 2	(2, 4)	(3, 3)	**Bitir**	(2, 4)
3	Oyuncu 1	(3, 1)	(2, 4)	**Bitir**	(3, 1)
2	Oyuncu 2	(0, 2)	(3, 1)	**Bitir**	(0, 2)
1	Oyuncu 1	(1, 0)	(0, 2)	**Bitir**	(1, 0)

1. düğüm, Oyuncu 1: Bitir → 1, Devam → 0 (devamın değeri (0, 2)). 1 > 0 ⇒ **Bitir**; düğümün değeri (1, 0).

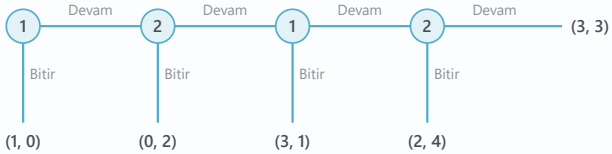
Kırkayak: Geriye Doğru Çıkarım Adım Adım



Düğüm	Karar veren	Bitir	Devamın değeri	Karar	Düğüm değeri
4	Oyuncu 2	(2, 4)	(3, 3)	**Bitir**	(2, 4)
3	Oyuncu 1	(3, 1)	(2, 4)	**Bitir**	(3, 1)
2	Oyuncu 2	(0, 2)	(3, 1)	**Bitir**	(0, 2)
1	Oyuncu 1	(1, 0)	(0, 2)	**Bitir**	(1, 0)

Alt oyun mükemmel denge: her oyuncu **her düğümde Bitir**. Oyun ilk hamlede (1, 0) ile biter; oysa (3, 3) ikisi için de daha iyiydi.

Kod İzleme: `geri_cikarim(dugumler, son)`



deger, oyunun sonundaki kazançla başlar: (3, 3).

Adım 1 / 14

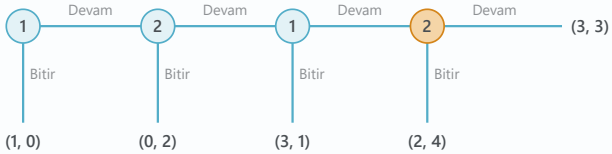
```
1  def geri_cikarim(dugumler, son):
▶     deger = son  # oyunun sonu
3     for k in reversed(range(len(dugumler))):
4       oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5       if bitir[oyuncu] >= deger[oyuncu]:
6         deger = bitir
7         print(k + 1, "Bitir")
8       else:
9         print(k + 1, "Devam")
10    return deger
```

DEĞİŞKENLER

`deger = (3, 3)`

ÇIKTI

Kod İzleme: `geri_cikarim(dugumler, son)`



k = 3 → 4. düğüm. Karar veren oyuncu indeksi 1 (Oyuncu 2); bitirirse (2, 4).

Adım 2 / 14

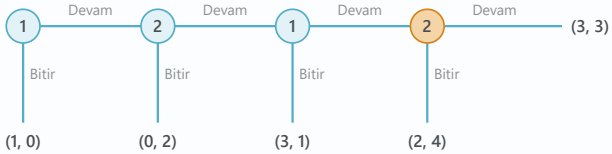
```
1  def geri_cikarim(dugumler, son):
2      deger = son  # oyunun sonu
▶      for k in reversed(range(len(dugumler))):
▶          oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5          if bitir[oyuncu] >= deger[oyuncu]:
6              deger = bitir
7              print(k + 1, "Bitir")
8          else:
9              print(k + 1, "Devam")
10     return deger
```

DEĞİŞKENLER

`k = 3` `oyuncu = 1` `bitir = (2, 4)` `deger = (3, 3)`

ÇIKTI

Kod İzleme: `geri_cikarim(dugumler, son)`



Koşul: bitir[1] = 4 ≥ deger[1] = 3 ? → **Doğru**

Adım 3 / 14

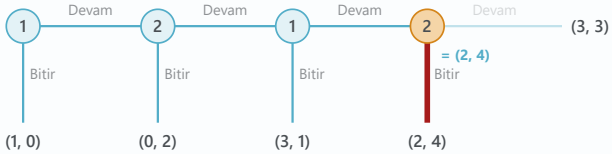
```
1  def geri_cikarim(dugumler, son):
2      deger = son  # oyunun sonu
3      for k in reversed(range(len(dugumler))):
4          oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
▶          if bitir[oyuncu] >= deger[oyuncu]:
6              deger = bitir
7              print(k + 1, "Bitir")
8          else:
9              print(k + 1, "Devam")
10     return deger
```

DEĞİŞKENLER

`k = 3` `oyuncu = 1` `bitir = (2, 4)` `deger = (3, 3)`

ÇIKTI

Kod İzleme: `geri_cikarim(dugumler, son)`



Bitirmek en az devam kadar iyi: deger = (2, 4) (4. düğümün değeri).

Adım 4 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5      if bitir[oyuncu] >= deger[oyuncu]:
▶        deger = bitir
▶        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

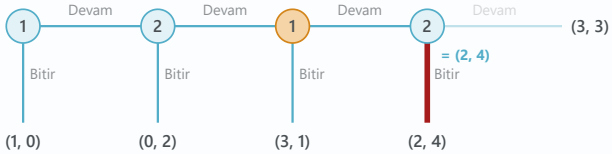
DEĞİŞKENLER

`k = 3` `oyuncu = 1` `bitir = (2, 4)` `deger = (2, 4)`

ÇIKTI

```
4 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



k = 2 → 3. düğüm. Karar veren oyuncu indeksi 0 (Oyuncu 1); bitirirse (3, 1).

Adım 5 / 14

```
1  def geri_cikarim(dugumler, son):
2      deger = son  # oyunun sonu
▶      for k in reversed(range(len(dugumler))):
▶          oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5          if bitir[oyuncu] >= deger[oyuncu]:
6              deger = bitir
7              print(k + 1, "Bitir")
8          else:
9              print(k + 1, "Devam")
10     return deger
```

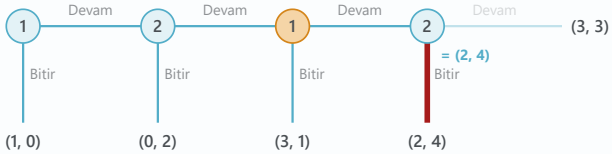
DEĞİŞKENLER

`k = 2` `oyuncu = 0` `bitir = (3, 1)` `deger = (2, 4)`

ÇIKTI

```
4 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Koşul: bitir[0] = 3 ≥ deger[0] = 2 ? → **Doğru**

Adım 6 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
▶      if bitir[oyuncu] >= deger[oyuncu]:
6        deger = bitir
7        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

DEĞİŞKENLER

`k = 2` `oyuncu = 0` `bitir = (3, 1)` `deger = (2, 4)`

ÇIKTI

```
4 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Bitirmek en az devam kadar iyi: deger = (3, 1) (3. düğümün değeri).

Adım 7 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5      if bitir[oyuncu] >= deger[oyuncu]:
▶        deger = bitir
▶        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

DEĞİŞKENLER

`k = 2` `oyuncu = 0` `bitir = (3, 1)` `deger = (3, 1)`

ÇIKTI

```
4 Bitir
3 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



k = 1 → 2. düğüm. Karar veren oyuncu indeksi 1 (Oyuncu 2); bitirirse (0, 2).

Adım 8 / 14

```
1  def geri_cikarim(dugumler, son):
2      deger = son  # oyunun sonu
▶      for k in reversed(range(len(dugumler))):
▶          oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5          if bitir[oyuncu] >= deger[oyuncu]:
6              deger = bitir
7              print(k + 1, "Bitir")
8          else:
9              print(k + 1, "Devam")
10     return deger
```

DEĞİŞKENLER

`k = 1` `oyuncu = 1`

`bitir = (0, 2)` `deger = (3, 1)`

ÇIKTI

```
4 Bitir
3 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Koşul: bitir[1] = 2 ≥ deger[1] = 1 ? → **Doğru**

Adım 9 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
▶      if bitir[oyuncu] >= deger[oyuncu]:
6        deger = bitir
7        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

DEĞİŞKENLER

`k = 1` `oyuncu = 1`

`bitir = (0, 2)` `deger = (3, 1)`

ÇIKTI

```
4 Bitir
3 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Bitirmek en az devam kadar iyi: deger = (0, 2) (2. düğümün değeri).

Adım 10 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5      if bitir[oyuncu] >= deger[oyuncu]:
▶        deger = bitir
▶        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

DEĞİŞKENLER

`k = 1` `oyuncu = 1`

`bitir = (0, 2)` `deger = (0, 2)`

ÇIKTI

```
4 Bitir
3 Bitir
2 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



k = 0 → 1. düğüm. Karar veren oyuncu indeksi 0 (Oyuncu 1); bitirirse (1, 0).

Adım 11 / 14

```
1  def geri_cikarim(dugumler, son):
2      deger = son  # oyunun sonu
▶      for k in reversed(range(len(dugumler))):
▶          oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5          if bitir[oyuncu] >= deger[oyuncu]:
6              deger = bitir
7              print(k + 1, "Bitir")
8          else:
9              print(k + 1, "Devam")
10     return deger
```

DEĞİŞKENLER

`k = 0` `oyuncu = 0` `bitir = (1, 0)` `deger = (0, 2)`

ÇIKTI

```
4 Bitir
3 Bitir
2 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Koşul: bitir[0] = 1 ≥ deger[0] = 0 ? → **Doğru**

Adım 12 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
▶      if bitir[oyuncu] >= deger[oyuncu]:
6        deger = bitir
7        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

DEĞİŞKENLER

`k = 0` `oyuncu = 0` `bitir = (1, 0)` `deger = (0, 2)`

ÇIKTI

```
4 Bitir
3 Bitir
2 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Bitirmek en az devam kadar iyi: deger = (1, 0) (1. düğümün değeri).

Adım 13 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5      if bitir[oyuncu] >= deger[oyuncu]:
▶        deger = bitir
▶        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

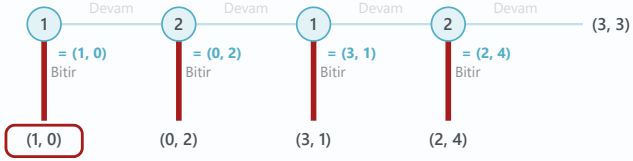
DEĞİŞKENLER

`k = 0` `oyuncu = 0` `bitir = (1, 0)` `deger = (1, 0)`

ÇIKTI

```
4 Bitir
3 Bitir
2 Bitir
1 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Döngü köke ulaştı: fonksiyon (1, 0) döndürür — alt oyun mükemmel dengenin sonucu.

Adım 14 / 14

```
1 def geri_cikarim(dugumler, son):
2   deger = son  # oyunun sonu
3   for k in reversed(range(len(dugumler))):
4     oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5     if bitir[oyuncu] >= deger[oyuncu]:
6       deger = bitir
7       print(k + 1, "Bitir")
8     else:
9       print(k + 1, "Devam")
▶   return deger
```

DEĞİŞKENLER

`deger = (1, 0)` `sonuç = (1, 0)`

ÇIKTI

```
4 Bitir
3 Bitir
2 Bitir
1 Bitir
```

Kırkayak Oyunu: Teori ve Deney

Teori (SPE)

Her oyuncu her düğümde Bitir der; oyun ilk hamlede (1, 0) ile biter. Oyunun herhangi bir Nash dengesinde de sonuç aynıdır.

Verimsizlik

(3, 3) her iki oyuncu için de (1, 0)'dan iyidir: SPE sonucu **Pareto optimal değildir**. Düğüm sayısı arttıkça kayıp büyür.

Deney

Laboratuvarda oyuncuların çoğu ilk düğümde bitirmez; birkaç tur devam eder (McKelvey ve Palfrey, 1992).

Neden?

Geriye doğru çıkarım, rasyonelliğin **ortak bilgi** olmasını ve uzun bir akıl yürütme zincirini gerektirir; rakibin biraz "iyi niyetli" olma ihtimali bile devam etmeyi kârlı kılar.

Kırkayak, sonlu tekrarlanan Mahkûmun İkilemi ile aynı mantığı paylaşır: son turdan geriye bakınca işbirliği çözülür. Sonsuz ufukta ise işbirliği dengede kalabilir.

Sık Yapılan Hatalar: Sıralı Oyunlar

Kökten başlamak
İlk hamle, sonraki hamlelere verilecek tepkiye bağlıdır; çözüm her zaman son karar düğümlerinden başlar.

Yanlış kişinin kazancı
Her düğümde yalnız *orada karar veren* oyuncunun sayısı karşılaştırılır (Bora'nın düğümünde ikinci sayı).

Stratejiyi yol sanmak
Strateji, ulaşılmayan düğümler dahil her düğüm için seçimi içerir: Ayşe'nin stratejisi (Yatır, Dava aç).

Nash = alt oyun mükemmel sanmak
Her alt oyun mükemmel denge Nash dengesidir; tersi doğru değil: inandırıcı olmayan tehdit içeren Nash dengeleri vardır.

Eşzamanlıyı sıralı çözmek
Hamleler görülmüyorsa (Mahkûmun İkilemi) ağaçta bilgi kümesi vardır; geriye doğru çıkarım doğrudan uygulanmaz, matris çözülür.

Söz = taahhüt sanmak
"Dava açarım" demek yetmez; tehdidin uygulanması o anda kârlı olmalı ya da önceden bağlanmalı (sözleşme, teminat).

Oyuna Dönüş: Connect Four

Kuramın cevabı

Oyun sonlu, mükemmel bilgili ve şanssızdır; Zermelo teoremine göre sonucu önceden belirlidir. 1988'de iki ayrı çalışma (James D. Allen; Victor Allis) cevabı buldu: ilk oyuncu ortadaki sütundan başlar ve hatasız oynarsa **kazanır**.

Gerçekte: ağaç çok büyük

En çok 42 hamle, hamle başına en çok 7 seçenek: $7^{42} \approx 3{,}1 \cdot 10^{35}$ hamle dizisi. Oyuncular ağacın yalnızca birkaç hamle ilerisine bakar; oyundaki "Hard" düzey 7 hamle derinlikte minimaks ve alfa-beta budaması kullanır, gerisini sezgisel puanla tahmin eder.

Oyun	Mükemmel oyunda sonuç	Nasıl biliniyor?
XOX (3×3)	beraberlik	tüm ağaç taranabilir
Connect Four (7×6)	ilk oyuncu kazanır	bilgisayar araması (1988)
Hex (11×11)	ilk oyuncu kazanır	strateji çalma argümanı; kazanan strateji bilinmiyor
Satranç	bilinmiyor	ağaç çözülemeyecek kadar büyük

Kırkayaktaki gibi: kuram "sonuç belli" der, ama uzun çıkarım zinciri tamamlanamadığı için oyun gerçekte ilginç kalır.

ÖZET

Özet

Algoritmik bakış

Hesaplama, verimlilik (Pareto, Anarşinin Bedeli), tasarım ve dinamik: dört soru.

Sınıflandırma

Zamanlama, işbirliği, toplam kazanç, simetri, bilgi (mükemmel / tam) ve ufuk birbirinden bağımsız boyutlardır.

Saf denge

En iyi tepkileri işaretle; iki işaretli hücre saf Nash dengesidir. Yazı-Tura'da yok, koordinasyon oyunlarında iki tane.

Klasik oyunlar

Çatışma oyunlarında (Yazı-Tura, Tavuk) çıkarlar zıttır; koordinasyon oyunlarında (Cinsiyetler Savaşı, Geyik Avı, Saf Koordinasyon, Bedava Para) sorun buluşmaktır.

Denge seçimi

Çoklu denge varsa Pareto, risk, odak noktası ve zayıf/kesin denge ayrımı hangisinin oynanacağını etkiler.

Geriye doğru çıkarım

Sıralı oyunları yapraktan köke çözer, alt oyun mükemmel dengeyi verir ve inandırıcı olmayan tehditleri eler.

ÖZET

Etkileşimli Oyunlar

Bu bölümün oyunları iki kişilik, sıralı, **mükemmel bilgili**, sonlu ve (kazan/kaybet açısından) sıfır toplamlıdır: kuramsal olarak geriye doğru çıkarımla çözülebilirler, ama ağaç çok büyüktür.

Hex

Beraberlik yoktur; strateji çalma argümanıyla ilk oyuncunun kazanan stratejisi olduğu kanıtlanır.

Connect Four

1988'de çözüldü: mükemmel oyunda ilk oyuncu kazanır.

Othello (Reversi)

8×8 tahtada mükemmel oyunun beraberlikle bittiği 2023'te hesaplamayla gösterildi. Taş sayısından çok hareketlilik (yasal hamle sayısı) önemlidir.

Tahta büyüdükçe bu oyunlar hesaplama açısından zorlaşır: genelleştirilmiş n×n Hex PSPACE-tamdır (Reisch, 1981). Bu tür sonuçların derlemesi: Hearn ve Demaine, *Games, Puzzles, and Computation*.

ÖZET

Örnek Problemler

1. (A, A) = (4, 1), (B, B) = (1, 4), diğer hücreler (0, 0). Saf dengeler hangileri? Oyun hangi klasik oyuna benzer?
2. Altı oyundan hangileri sıfır toplamlı, hangileri simetrik?
3. Kırkayak'ta en sağdaki kazanç (5, 5) olsaydı alt oyun mükemmel denge?
4. Piyasaya girişte (Girme, Savaş) neden alt oyun mükemmel değil?

▶ **Cevapları göster**

Kaynaklar

- M. J. Osborne, *An Introduction to Game Theory*, Oxford University Press, 2004 (Bölüm 2–5).
- N. Nisan, T. Roughgarden, É. Tardos, V. Vazirani (ed.), *Algorithmic Game Theory*, Cambridge University Press, 2007 (Bölüm 1).
- Y. Shoham, K. Leyton-Brown, *Multiagent Systems*, Cambridge University Press, 2009 (Bölüm 3 ve 5).
- R. W. Rosenthal, "Games of perfect information, predatory pricing and the chain-store paradox", *J. Economic Theory* 25, 1981.
- R. D. McKelvey, T. R. Palfrey, "An experimental study of the centipede game", *Econometrica* 60(4), 1992.
- B. Skyrms, *The Stag Hunt and the Evolution of Social Structure*, Cambridge University Press, 2004.
- A. R. Karlin, Y. Peres, *Game Theory, Alive*, American Mathematical Society, 2017 (2×2 oyunlar, karışık denge).
- R. A. Hearn, E. D. Demaine, *Games, Puzzles, and Computation*, A K Peters, 2009 (Hex gibi oyunların karmaşıklığı).
- S. Reisch, "Hex ist PSPACE-vollständig", *Acta Informatica* 15, 1981.
- Ders sayfası: T. Roughgarden, CS364A Algorithmic Game Theory (Stanford).
- Araçlar: Gambit (oyun ağaçları ve matris oyunlarında denge hesabı) · Nashpy (iki oyunculu oyunlar için Python kütüphanesi).

Son

Bölüm 2: Oyun Kuramına Giriş

Sonraki bölüm: Bölüm 3: Nash Dengesi

Çalışma soruları · Terim sözlüğü